USE CLAUDE CODE

Best practices for Claude Code

Tips and patterns for getting the most out of Claude Code, from configuring your environment to scaling across parallel sessions.

Claude Code is an agentic coding environment. Unlike a chatbot that answers questions and waits, Claude Code can read your files, run commands, make changes, and autonomously work through problems while you watch, redirect, or step away entirely.

This changes how you work. Instead of writing code yourself and asking Claude to review it, you describe what you want and Claude figures out how to build it. Claude explores, plans, and implements.

But this autonomy still comes with a learning curve. Claude works within certain constraints you need to understand.

This guide covers patterns that have proven effective across Anthropic's internal teams and for engineers using Claude Code across various codebases, languages, and environments. For how the agentic loop works under the hood, see **How Claude Code works**.

Most best practices are based on one constraint: Claude's context window fills up fast, and performance degrades as it fills.

Claude's context window holds your entire conversation, including every message, every file Claude reads, and every command output. However, this can fill up fast. A single debugging session or codebase exploration might generate and consume tens of thousands of tokens.

This matters since LLM performance degrades as context fills. When the context window is getting full, Claude may start "forgetting" earlier instructions or making more mistakes. The context window is the most important resource to manage. To see how a session fills up in practice, **watch an interactive walkthrough** of what loads at startup and what each file read costs. Track context usage continuously with a **custom status line**, and see **Reduce token usage** for strategies on reducing

token usage.

Give Claude a way to verify its work

> Give Claude a check it can run: tests, a build, a screenshot to compare. It's the difference between a session you watch and one you walk away from.

Claude stops when the work looks done. Without a check it can run, "looks done" is the only signal available, and you become the verification loop: every mistake waits for you to notice it. Give Claude something that produces a pass or fail, and the loop closes on its own. Claude does the work, runs the check, reads the result, and iterates until the check passes.

The check is anything that returns a signal Claude can read in the conversation: a test suite, a build exit code, a linter, a script that diffs output against a fixture, or a **browser screenshot** compared against a design.

Strategy	Before	After
Provide verification criteria	*"implement a function that validates email addresses"*	*"write a validateEmail function. example test cases: **user@example.com** is true, invalid is false, **user@.com** is false. run the tests after implementing"*
Verify UI changes visually	*"make the dashboard look better"*	*"[paste screenshot] implement this design. take a screenshot of the result and compare it to the original. list differences and fix them"*
Address root causes, not symptoms	*"the build is failing"*	*"the build fails with this error: [paste error]. fix it and verify the build succeeds. address the root cause, don't suppress the error"*

Once the check exists, decide how hard it gates the stop:

- **In one prompt**: ask Claude to run the check and iterate in the same message, as in the table above.
- **Across a session**: set the check as a `/goal` **condition**. A separate evaluator re-checks it after every turn and Claude keeps working until it holds.
- **As a deterministic gate**: a **Stop hook** runs your check as a script and blocks the turn from ending until it passes. Claude Code overrides the hook and ends the turn after 8 consecutive blocks.
- **By a second opinion**: a **verification subagent** or a **dynamic workflow** that checks its own

findings has a fresh model try to refute the result, so the agent doing the work isn't the one grading it.

Each step trades setup for attention. The prompt version works on any task today. The `/goal` and Stop hook versions are what let an unattended run finish correctly without you.

Have Claude show evidence rather than asserting success: the test output, the command it ran and what it returned, or a screenshot of the result. Reviewing evidence is faster than re-running the verification yourself, and it works for sessions you weren't watching.

Explore first, then plan, then code

Separate research and planning from implementation to avoid solving the wrong problem.

Letting Claude jump straight to coding can produce code that solves the wrong problem. Use **plan mode** to separate exploration from execution.

The recommended workflow has four phases:

1 Explore

Enter plan mode. Claude reads files and answers questions without making changes.

claude (plan mode)

```
read /src/auth and understand how we handle sessions and login.
also look at how we manage environment variables for secrets.
```

2 Plan

Ask Claude to create a detailed implementation plan.

claude (plan mode)

```
I want to add Google OAuth. What files need to change?
What's the session flow? Create a plan.
```

Press `Ctrl+G` to open the plan in your text editor for direct editing before Claude proceeds.

3 Implement

Switch out of plan mode and let Claude code, verifying against its plan.

claude (default mode)

```
implement the OAuth flow from your plan. write tests for the
callback handler, run the test suite and fix any failures.
```

4 Commit

Ask Claude to commit with a descriptive message and create a PR.

claude (default mode)

```
commit with a descriptive message and open a PR
```

Plan mode is useful, but also adds overhead.

For tasks where the scope is clear and the fix is small (like fixing a typo, adding a log line, or renaming a variable) ask Claude to do it directly.

Planning is most useful when you're uncertain about the approach, when the change modifies multiple files, or when you're unfamiliar with the code being modified. If you could describe the diff in one sentence, skip the plan.

Provide specific context in your prompts

The more precise your instructions, the fewer corrections you'll need.

Claude can infer intent, but it can't read your mind. Reference specific files, mention constraints, and point to example patterns.

Strategy	Before	After
Scope the task. Specify which file, what scenario, and testing preferences.	*”add tests for foo.py”*	*”write a test for foo.py covering the edge case where the user is logged out. avoid mocks.”*
Point to sources. Direct Claude to the source that can answer a question.	*”why does ExecutionFactory have such a weird api?”*	*”look through ExecutionFactory’s git history and summarize how its api came to be”*
Reference existing patterns. Point Claude to patterns in your codebase.	*”add a calendar widget”*	*”look at how existing widgets are implemented on the home page to understand the patterns. HotDogWidget.php is a good example. follow the pattern to implement a new calendar widget that lets the user select a month and paginate forwards/backwards to pick a year. build from scratch without libraries other than the ones already used in the codebase.”*
Describe the symptom. Provide the symptom, the likely location, and what “fixed” looks like.	*”fix the login bug”*	*”users report that login fails after session timeout. check the auth flow in src/auth/, especially token refresh. write a failing test that reproduces the issue, then fix it”*

Vague prompts can be useful when you’re exploring and can afford to course-correct. A prompt like `"what would you improve in this file?"` can surface things you wouldn’t have thought to ask about.

Provide rich content

Use `@` to reference files, paste screenshots/images, or pipe data directly.

You can provide rich data to Claude in several ways:

- **Reference files with** `@` instead of describing where code lives. Claude reads the file before responding.
- **Paste images directly**. Copy/paste or drag and drop images into the prompt.
- **Give URLs** for documentation and API references. Use `/permissions` to allowlist frequently-used domains.
- **Pipe in data** by running `cat error.log | claude` to send file contents directly.
- **Let Claude fetch what it needs**. Tell Claude to pull context itself using Bash commands, MCP tools, or by reading files.

Configure your environment

A few setup steps make Claude Code significantly more effective across all your sessions. For a full overview of extension features and when to use each one, see **Extend Claude Code**.

Write an effective CLAUDE.md

> Run `/init` to generate a starter CLAUDE.md file based on your current project structure, then refine over time.

CLAUDE.md is a special file that Claude reads at the start of every conversation. Include Bash commands, code style, and workflow rules. This gives Claude persistent context it can't infer from code alone.

The `/init` command analyzes your codebase to detect build systems, test frameworks, and code patterns, giving you a solid foundation to refine.

There's no required format for CLAUDE.md files, but keep it short and human-readable. For example:

CLAUDE.md

```
# Code style
- Use ES modules (import/export) syntax, not CommonJS (require)
- Destructure imports when possible (eg. import { foo } from 'bar')

# Workflow
- Be sure to typecheck when you're done making a series of code changes
- Prefer running single tests, and not the whole test suite, for performance
```

CLAUDE.md is loaded every session, so only include things that apply broadly. For domain knowledge or workflows that are only relevant sometimes, use **skills** instead. Claude loads them on demand without bloating every conversation.

Keep it concise. For each line, ask: *"Would removing this cause Claude to make mistakes?"* If not, cut it. Bloated CLAUDE.md files cause Claude to ignore your actual instructions!

✅ Include	❌ Exclude
Bash commands Claude can't guess	Anything Claude can figure out by reading code
Code style rules that differ from defaults	Standard language conventions Claude already knows
Testing instructions and preferred test runners	Detailed API documentation (link to docs instead)
Repository etiquette (branch naming, PR conventions)	Information that changes frequently
Architectural decisions specific to your project	Long explanations or tutorials
Developer environment quirks (required env vars)	File-by-file descriptions of the codebase
Common gotchas or non-obvious behaviors	Self-evident practices like "write clean code"

If Claude keeps doing something you don't want despite having a rule against it, the file is probably too long and the rule is getting lost. If Claude asks you questions that are answered in CLAUDE.md, the phrasing might be ambiguous. Treat CLAUDE.md like code: review it when things go wrong, prune it regularly, and test changes by observing whether Claude's behavior actually shifts.

You can tune instructions by adding emphasis (e.g., "IMPORTANT" or "YOU MUST") to improve adherence. Check CLAUDE.md into git so your team can contribute. The file compounds in value over time.

CLAUDE.md files can import additional files using `@path/to/import` syntax:

CLAUDE.md

```
See @README.md for project overview and @package.json for available npm
commands.

# Additional Instructions
- Git workflow: @docs/git-instructions.md
- Personal overrides: @~/.claude/my-project-instructions.md
```

You can place CLAUDE.md files in several locations:

- **Home folder (** `~/.claude/CLAUDE.md` **)**: applies to all Claude sessions
- **Project root (** `./CLAUDE.md` **)**: check into git to share with your team
- **Project root (** `./CLAUDE.local.md` **)**: personal project-specific notes; add this file to your `.gitignore` so it isn't shared with your team
- **Parent directories**: useful for monorepos where both `root/CLAUDE.md` and

`root/foo/CLAUDE.md` are pulled in automatically

- **Child directories**: Claude pulls in child CLAUDE.md files on demand when it reads a file in those directories

Configure permissions

> Use **auto mode** to let a classifier handle approvals, `/permissions` to allowlist specific commands, or `/sandbox` for OS-level isolation. Each reduces interruptions while keeping you in control.

By default, Claude Code requests permission for actions that might modify your system: file writes, Bash commands, MCP tools, etc. This is safe but tedious. After the tenth approval you're not really reviewing anymore, you're just clicking through. There are three ways to reduce these interruptions:

- **Auto mode**: a separate classifier model reviews commands and blocks only what looks risky: scope escalation, unknown infrastructure, or hostile-content-driven actions. Best when you trust the general direction of a task but don't want to click through every step
- **Permission allowlists**: permit specific tools you know are safe, like `npm run lint` or `git commit`
- **Sandboxing**: enable OS-level isolation that restricts filesystem and network access, allowing Claude to work more freely within defined boundaries

Read more about **permission modes**, **permission rules**, and **sandboxing**.

Use CLI tools

> Tell Claude Code to use CLI tools like `gh`, `aws`, `gcloud`, and `sentry-cli` when interacting with external services.

CLI tools are the most context-efficient way to interact with external services. If you use GitHub, install the `gh` CLI. Claude knows how to use it for creating issues, opening pull requests, and reading comments. Without `gh`, Claude can still use the GitHub API, but unauthenticated requests often hit rate limits.

Claude is also effective at learning CLI tools it doesn't already know. Try prompts like `Use 'foo-cli-tool --help' to learn about foo tool, then use it to solve A, B, C.`

Connect MCP servers

Run `claude mcp add` to connect external tools like Notion, Figma, or your database.

With **MCP servers**, you can ask Claude to implement features from issue trackers, query databases, analyze monitoring data, integrate designs from Figma, and automate workflows.

Set up hooks

Use hooks for actions that must happen every time with zero exceptions.

Hooks run scripts automatically at specific points in Claude's workflow. Unlike CLAUDE.md instructions which are advisory, hooks are deterministic and guarantee the action happens.

Claude can write hooks for you. Try prompts like *"Write a hook that runs eslint after every file edit"* or *"Write a hook that blocks writes to the migrations folder."* Edit `.claude/settings.json` directly to configure hooks by hand, and run `/hooks` to browse what's configured.

Create skills

Create `SKILL.md` files in `.claude/skills/` to give Claude domain knowledge and reusable workflows.

Skills extend Claude's knowledge with information specific to your project, team, or domain. Claude applies them automatically when relevant, or you can invoke them directly with `/skill-name`.

Create a skill by adding a directory with a `SKILL.md` to `.claude/skills/`:

.claude/skills/api-conventions/SKILL.md

```
---
name: api-conventions
description: REST API design conventions for our services
---
# API Conventions
- Use kebab-case for URL paths
- Use camelCase for JSON properties
- Always include pagination for list endpoints
- Version APIs in the URL path (/v1/, /v2/)
```

Skills can also define repeatable workflows you invoke directly:

.claude/skills/fix-issue/SKILL.md

```
---
name: fix-issue
description: Fix a GitHub issue
disable-model-invocation: true
---
Analyze and fix the GitHub issue: $ARGUMENTS.

1. Use `gh issue view` to get the issue details
2. Understand the problem described in the issue
3. Search the codebase for relevant files
4. Implement the necessary changes to fix the issue
5. Write and run tests to verify the fix
6. Ensure code passes linting and type checking
7. Create a descriptive commit message
8. Push and create a PR
```

Run `/fix-issue 1234` to invoke it. Use `disable-model-invocation: true` for workflows with side effects that you want to trigger manually.

Create custom subagents

Define specialized assistants in `.claude/agents/` that Claude can delegate to for isolated tasks.

Subagents run in their own context with their own set of allowed tools. They're useful for tasks that

read many files or need specialized focus without cluttering your main conversation.

.claude/agents/security-reviewer.md

```
---
name: security-reviewer
description: Reviews code for security vulnerabilities
tools: Read, Grep, Glob, Bash
model: opus
---
You are a senior security engineer. Review code for:
- Injection vulnerabilities (SQL, XSS, command injection)
- Authentication and authorization flaws
- Secrets or credentials in code
- Insecure data handling

Provide specific line references and suggested fixes.
```

Tell Claude to use subagents explicitly: *"Use a subagent to review this code for security issues."*

Install plugins

Run `/plugin` to browse the marketplace. Plugins add skills, tools, and integrations without configuration.

Plugins bundle skills, hooks, subagents, and MCP servers into a single installable unit from the community and Anthropic. If you work with a typed language, install a **code intelligence plugin** to give Claude precise symbol navigation and automatic error detection after edits.

For guidance on choosing between skills, subagents, hooks, and MCP, see **Extend Claude Code**.

Communicate effectively

The way you communicate with Claude Code significantly impacts the quality of results.

Ask codebase questions

> Ask Claude questions you'd ask a senior engineer.

When onboarding to a new codebase, use Claude Code for learning and exploration. You can ask Claude the same sorts of questions you would ask another engineer:

- How does logging work?
- How do I make a new API endpoint?
- What does `async move { ... }` do on line 134 of `foo.rs`?
- What edge cases does `CustomerOnboardingFlowImpl` handle?
- Why does this code call `foo()` instead of `bar()` on line 333?

Using Claude Code this way is an effective onboarding workflow, improving ramp-up time and reducing load on other engineers. No special prompting required: ask questions directly.

Let Claude interview you

> For larger features, have Claude interview you first. Start with a minimal prompt and ask Claude to interview you using the `AskUserQuestion` tool.

Claude asks about things you might not have considered yet, including technical implementation, UI/UX, edge cases, and tradeoffs.

```
I want to build [brief description]. Interview me in detail using
the AskUserQuestion tool.

Ask about technical implementation, UI/UX, edge cases, concerns,
and tradeoffs. Don't ask obvious questions, dig into the hard
parts I might not have considered.

Keep interviewing until we've covered everything, then write a
complete spec to SPEC.md.
```

Once the spec is complete, start a fresh session to execute it. The new session has clean context focused entirely on implementation, and you have a written spec to reference.

The most useful specs are self-contained: they name the files and interfaces involved, state what is

out of scope, and end with an end-to-end verification step that proves the feature works. Time spent making the spec precise pays off more than time spent watching the implementation.

Manage your session

Conversations are persistent and reversible. Use this to your advantage!

Course-correct early and often

> Correct Claude as soon as you notice it going off track.

The best results come from tight feedback loops. Though Claude occasionally solves problems perfectly on the first attempt, correcting it quickly generally produces better solutions faster.

- `Esc`: stop Claude mid-action with the `Esc` key. Context is preserved, so you can redirect.
- `Esc + Esc` **or** `/rewind`: press `Esc` twice or run `/rewind` to open the rewind menu and restore previous conversation and code state, or summarize from a selected message.
- `"Undo that"`: have Claude revert its changes.
- `/clear`: reset context between unrelated tasks. Long sessions with irrelevant context can reduce performance.

If you've corrected Claude more than twice on the same issue in one session, the context is cluttered with failed approaches. Run `/clear` and start fresh with a more specific prompt that incorporates what you learned. A clean session with a better prompt almost always outperforms a long session with accumulated corrections.

Manage context aggressively

> Run `/clear` between unrelated tasks to reset context.

Claude Code automatically compacts conversation history when you approach context limits, which preserves important code and decisions while freeing space.

During long sessions, Claude's context window can fill with irrelevant conversation, file contents, and

commands. This can reduce performance and sometimes distract Claude.

- Use `/clear` frequently between tasks to reset the context window entirely
- When auto compaction triggers, Claude summarizes what matters most, including code patterns, file states, and key decisions
- For more control, run `/compact <instructions>`, like `/compact Focus on the API changes`
- To compact only part of the conversation, use `Esc + Esc` or `/rewind`, select a message checkpoint, and choose **Summarize from here** or **Summarize up to here**. The first condenses messages from that point forward while keeping earlier context intact; the second condenses earlier messages while keeping recent ones in full. See **Restore vs. summarize**.
- Customize compaction behavior in CLAUDE.md with instructions like `"When compacting, always preserve the full list of modified files and any test commands"` to ensure critical context survives summarization
- For quick questions that don't need to stay in context, use `/btw`. The answer appears in a dismissible overlay and never enters conversation history, so you can check a detail without growing context.

Use subagents for investigation

> Delegate research with `"use subagents to investigate X"`. They explore in a separate context, keeping your main conversation clean for implementation.

Since context is your fundamental constraint, subagents are one of the most powerful tools available. When Claude researches a codebase it reads lots of files, all of which consume your context. Subagents run in separate context windows and report back summaries:

```
Use subagents to investigate how our authentication system
handles token
refresh, and whether we have any existing OAuth utilities I
should reuse.
```

The subagent explores the codebase, reads relevant files, and reports back with findings, all without cluttering your main conversation.

You can also use subagents for verification after Claude implements something:

```
use a subagent to review this code for edge cases
```

Rewind with checkpoints

> Every prompt you send creates a checkpoint. You can restore conversation, code, or both to any previous checkpoint.

Claude automatically snapshots files before each change so a checkpoint can restore them. Double-tap `Escape` or run `/rewind` to open the rewind menu. You can restore conversation only, restore code only, restore both, or summarize from a selected message. See **Checkpointing** for details.

Instead of carefully planning every move, you can tell Claude to try something risky. If it doesn't work, rewind and try a different approach. Checkpoints persist across sessions, so you can close your terminal and still rewind later.

> ⚠ Checkpoints only track changes made *by Claude*, not external processes. This isn't a replacement for git.

Resume conversations

> Name sessions with `/rename` and treat them like branches: each workstream gets its own persistent context.

Claude Code saves conversations locally, so when a task spans multiple sittings you don't have to re-explain the context. Run `claude --continue` to pick up the most recent session, or `claude --resume` to choose from a list. Give sessions descriptive names like `oauth-migration` so you can find them later. See **Manage sessions** for the full set of resume, branch, and naming controls.

Automate and scale

Once you're effective with one Claude, multiply your output with parallel sessions, non-interactive mode, and fan-out patterns.

Everything so far assumes one human, one Claude, and one conversation. But Claude Code scales horizontally. The techniques in this section show how you can get more done.

Run non-interactive mode

Use `claude -p "prompt"` in CI, pre-commit hooks, or scripts. Add `--output-format stream-json --verbose` for streaming JSON output.

With `claude -p "your prompt"`, you can run Claude non-interactively, without a session. **Non-interactive mode** is how you integrate Claude into CI pipelines, pre-commit hooks, or any automated workflow. The output formats let you parse results programmatically: plain text, JSON, or streaming JSON.

```
# One-off queries
claude -p "Explain what this project does"

# Structured output for scripts
claude -p "List all API endpoints" --output-format json

# Streaming for real-time processing
claude -p "Analyze this log file" --output-format stream-json --
verbose
```

Run multiple Claude sessions

Run multiple Claude sessions in parallel to speed up development, run isolated experiments, or start complex workflows.

Pick the parallel approach that fits how much coordination you want to do yourself:

- **Worktrees**: run separate CLI sessions in isolated git checkouts so edits don't collide
- **Desktop app**: manage multiple local sessions visually, each in its own worktree
- **Claude Code on the web**: run sessions on Anthropic-managed cloud infrastructure in isolated VMs
- **Agent teams**: automated coordination of multiple sessions with shared tasks, messaging, and a team lead

Beyond parallelizing work, multiple sessions enable quality-focused workflows. A fresh context improves code review since Claude won't be biased toward code it just wrote.

For example, use a Writer/Reviewer pattern:

Session A (Writer)	Session B (Reviewer)
`Implement a rate limiter for our API endpoints`	
	`Review the rate limiter implementation in @src/middleware/rateLimiter.ts. Look for edge cases, race conditions, and consistency with our existing middleware patterns.`
`Here's the review feedback: [Session B output]. Address these issues.`	

You can do something similar with tests: have one Claude write tests, then another write code to pass them.

Fan out across files

> Loop through tasks calling `claude -p` for each. Use `--allowedTools` to scope permissions for batch operations.

For large migrations or analyses, you can distribute work across many parallel Claude invocations:

1. **Generate a task list**

   Have Claude list all files that need migrating (e.g., `list all 2,000 Python files that need migrating`)

2. **Write a script to loop through the list**

```
for file in $(cat files.txt); do
  claude -p "Migrate $file from React to Vue. Return OK or
FAIL." \
    --allowedTools "Edit,Bash(git commit *)"
done
```

3. **Test on a few files, then run at scale**

> Refine your prompt based on what goes wrong with the first 2-3 files, then run on the full set. The `--allowedTools` flag restricts what Claude can do, which matters when you're running unattended.

You can also integrate Claude into existing data/processing pipelines:

```
claude -p "<your prompt>" --output-format json | your_command
```

Use `--verbose` for debugging during development, and turn it off in production.

Run autonomously with auto mode

For uninterrupted execution with background safety checks, use **auto mode**. A classifier model reviews commands before they run, blocking scope escalation, unknown infrastructure, and hostile-content-driven actions while letting routine work proceed without prompts.

```
claude --permission-mode auto -p "fix all lint errors"
```

For non-interactive runs with the `-p` flag, auto mode aborts if the classifier repeatedly blocks actions, since there is no user to fall back to. See **when auto mode falls back** for thresholds.

Add an adversarial review step

Before treating a task as done, have a subagent review the diff in a fresh context and report gaps.

The longer Claude works unattended, the more an independent check matters before you count the work as done. A reviewer running in a fresh **subagent** context sees only the diff and the criteria you give it, not the reasoning that produced the change, so it evaluates the result on its own terms.

For a correctness check, run the bundled **`/code-review` skill**, which reviews the current diff for bugs in a fresh subagent and returns findings to the session. To check the diff against your plan instead, write the review prompt yourself. Name the work to check, the plan to check it against, and what counts as a finding:

```
Use a subagent to review the rate limiter diff against PLAN.md.
Check that
every requirement is implemented, the listed edge cases have
tests, and
nothing outside the task's scope changed. Report gaps, not style
preferences.
```

Because the reviewer runs as a subagent, the implementing session receives the gaps directly and can fix them and re-review without you copying findings between windows. For longer autonomous runs, an **agent team** can keep this loop going across many tasks while you spot-check the recorded findings.

A reviewer prompted to find gaps will usually report some, even when the work is sound, because that is what it was asked to do. Chasing every finding leads to over-engineering: extra abstraction layers, defensive code, and tests for cases that can't happen. Tell the reviewer to flag only gaps that affect correctness or the stated requirements, and treat the rest as optional.

Avoid common failure patterns

These are common mistakes. Recognizing them early saves time:

- **The kitchen sink session.** You start with one task, then ask Claude something unrelated, then go back to the first task. Context is full of irrelevant information.

  > **Fix**: `/clear` between unrelated tasks.

- **Correcting over and over.** Claude does something wrong, you correct it, it's still wrong, you correct again. Context is polluted with failed approaches.

  > **Fix**: After two failed corrections, `/clear` and write a better initial prompt incorporating what you learned.

- **The over-specified CLAUDE.md.** If your CLAUDE.md is too long, Claude ignores half of it because important rules get lost in the noise.

  > **Fix**: Ruthlessly prune. If Claude already does something correctly without the instruction, delete it or convert it to a hook.

- **The trust-then-verify gap.** Claude produces a plausible-looking implementation that doesn't handle edge cases.

  > **Fix**: Always provide verification (tests, scripts, screenshots). If you can't verify it, don't ship it.

- **The infinite exploration.** You ask Claude to "investigate" something without scoping it. Claude reads hundreds of files, filling the context.

  > **Fix**: Scope investigations narrowly or use subagents so the exploration doesn't consume your main context.

Develop your intuition

The patterns in this guide aren't set in stone. They're starting points that work well in general, but might not be optimal for every situation.

Sometimes you *should* let context accumulate because you're deep in one complex problem and the history is valuable. Sometimes you should skip planning and let Claude figure it out because the task is exploratory. Sometimes a vague prompt is exactly right because you want to see how Claude interprets the problem before constraining it.

Pay attention to what works. When Claude produces great output, notice what you did: the prompt structure, the context you provided, the mode you were in. When Claude struggles, ask why. Was the context too noisy? The prompt too vague? The task too big for one pass?

Over time, you'll develop intuition that no guide can capture. You'll know when to be specific and when to be open-ended, when to plan and when to explore, when to clear context and when to let it accumulate.

Related resources

- **How Claude Code works**: the agentic loop, tools, and context management
- **Extend Claude Code**: skills, hooks, MCP, subagents, and plugins
- **Common workflows**: step-by-step recipes for debugging, testing, PRs, and more
- **CLAUDE.md**: store project conventions and persistent context